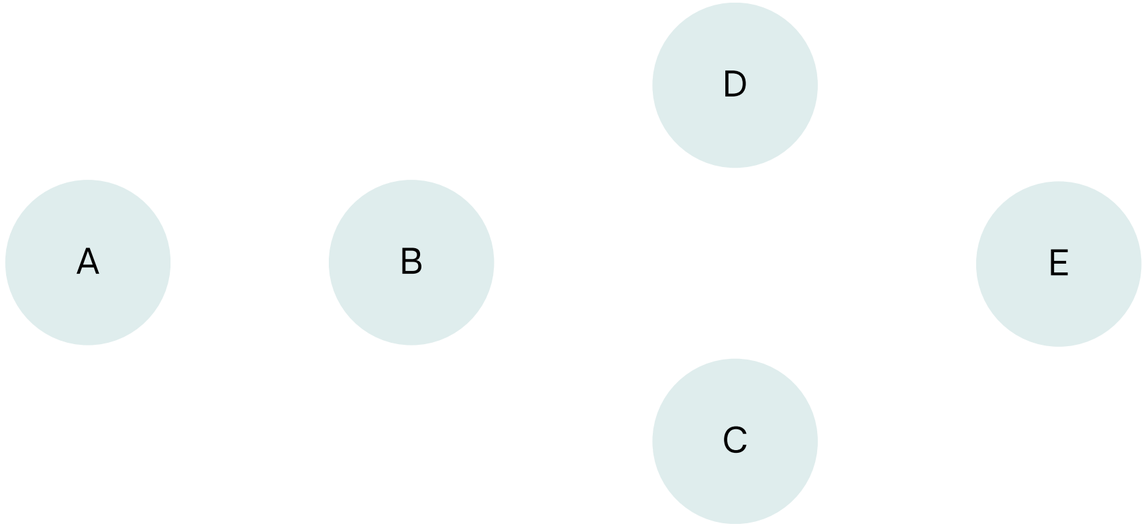
Minimum Steiner Tree

We begin with a well-known problem, that of a MIN-SPANNING-TREE, and then generalize from there.

Minimum Steiner Tree

Imagine you were given a map containing a set of cities, and were asked to develop a plan for connecting these cities with roads. Building a road costs N pesos per kilometer, and you want to minimize the length of the highways. Perhaps the map looks like this (drawn on a 2D plane, units: kilometers):



How do you connect the cities with a road network as cheaply as possible?

Minimum Steiner Tree

This is a standard network design question, and the solution is typically to find the minimum cost spanning tree. Recall that the problem of finding a MIN-SPANNING-TREE is defined as follows:

General Approach

Definition:

Given a weighted graph G = (V, E) the goal is to find a subset of edges E_0 ⊆ E such that:

- The subgraph (V, E_0) connects all vertices without cycles.
- The total edge weight is as small as possible.

Step 1

Compute the distance d(u,v) for every pair of locations (u,v), which takes $O(V^2)$ time.

Step 2

Construct a complete graph where each location is a vertex, and every pair of vertices is connected by an edge weighted by their distance.

Step 3

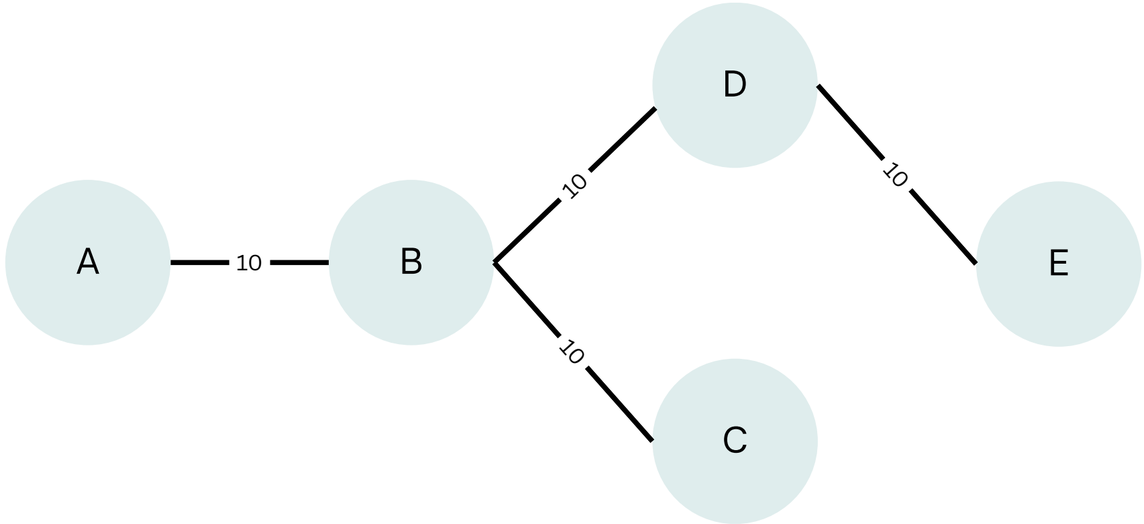
Apply an MST algorithm such as Kruskal's or Prim's to find the minimum spanning tree. For a complete graph, this takes $O(V^2 \log V)$ time.

Step 4

Return the resulting minimum spanning tree.

Minimum Steiner Tree

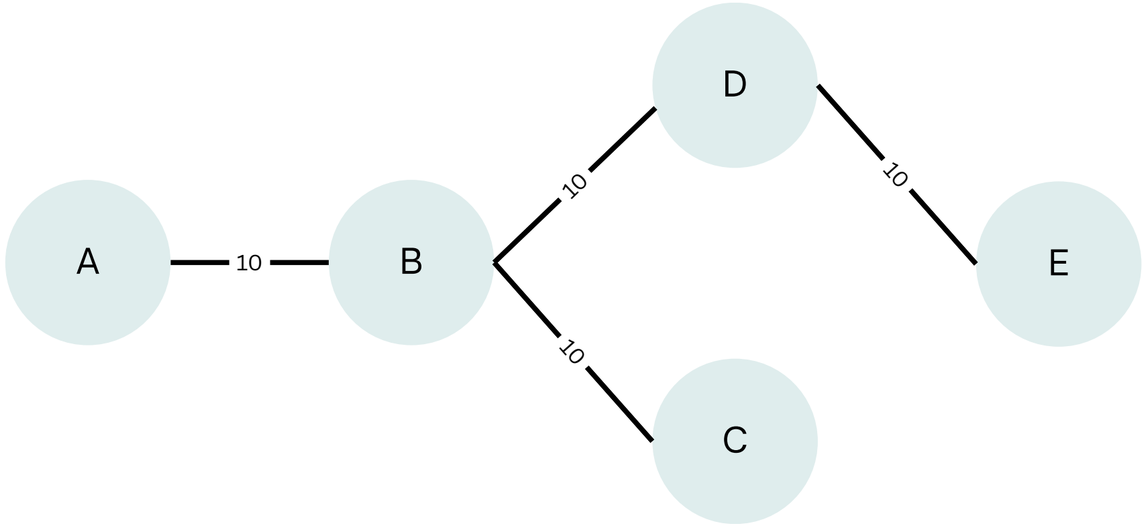
Imagine you were given a map containing a set of cities, and were asked to develop a plan for connecting these cities with roads. Building a road costs N pesos per kilometer, and you want to minimize the length of the highways. Perhaps the map looks like this (drawn on a 2D plane, units: kilometers):



Applying the standard approach for the MST we get a total cost of 40 Km to connect all cities.

Minimum Steiner Tree

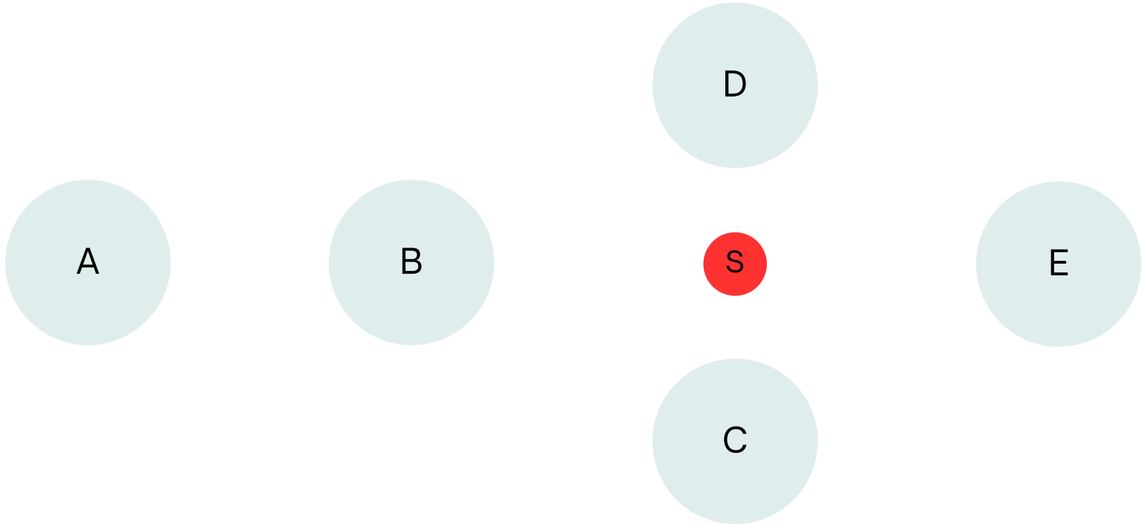
Imagine you were given a map containing a set of cities, and were asked to develop a plan for connecting these cities with roads. Building a road costs N pesos per kilometer, and you want to minimize the length of the highways. Perhaps the map looks like this (drawn on a 2D plane, units: kilometers):



What if we added an intersection in the network?

Minimum Steiner Tree

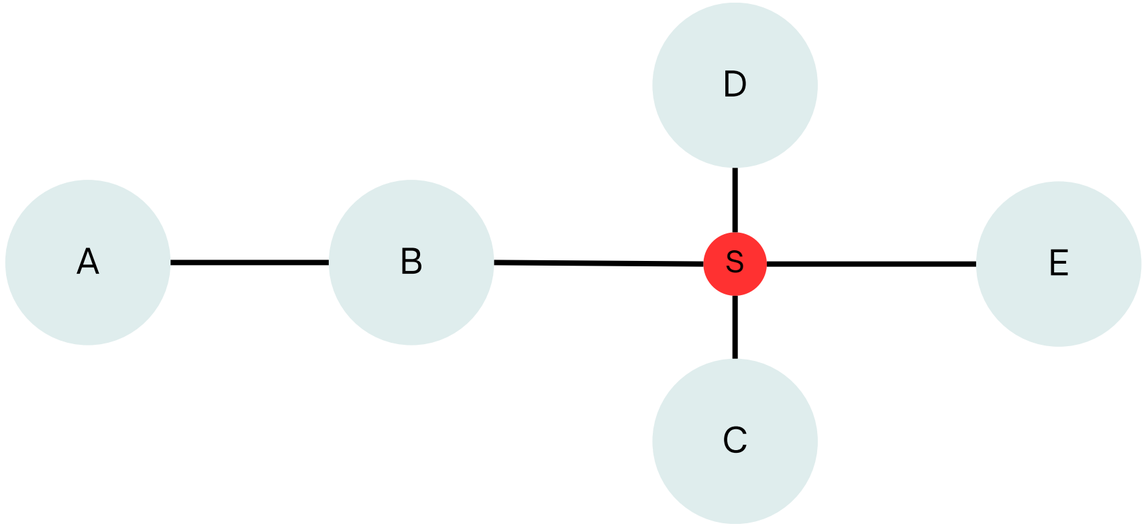
Imagine you were given a map containing a set of cities, and were asked to develop a plan for connecting these cities with roads. Building a road costs N pesos per kilometer, and you want to minimize the length of the highways. Perhaps the map looks like this (drawn on a 2D plane, units: kilometers):



What if we added an intersection in the network?

Minimum Steiner Tree

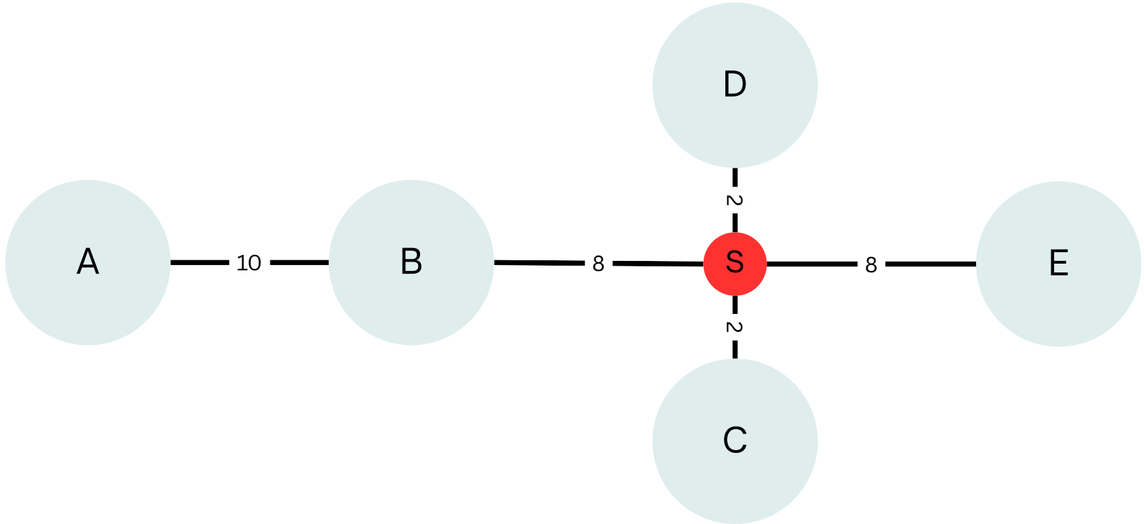
Imagine you were given a map containing a set of cities, and were asked to develop a plan for connecting these cities with roads. Building a road costs N pesos per kilometer, and you want to minimize the length of the highways. Perhaps the map looks like this (drawn on a 2D plane, units: kilometers):



What if we added an intersection in the network?

Minimum Steiner Tree

Imagine you were given a map containing a set of cities, and were asked to develop a plan for connecting these cities with roads. Building a road costs N pesos per kilometer, and you want to minimize the length of the highways. Perhaps the map looks like this (drawn on a 2D plane, units: kilometers):



By introducing an intersection, we can achieve a lower overall cost of 30 Km to connect all cities!

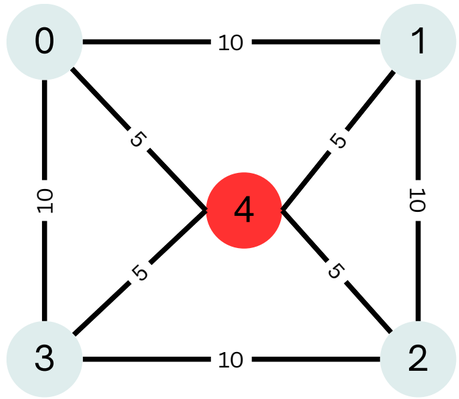
Minimum Steiner Tree

This perfectly sets up the core concept of the Steiner Tree.

The primary goal of a Steiner tree is to connect a given set of required nodes (terminal nodes) with the minimum total network length, possibly by adding extra nodes called Steiner nodes. These Steiner nodes are non-terminal points, such as intersections, that help reduce the overall path cost.

Brute Force Setup

Test Graph



= Steiner

= Terminal

Algorithm

1. Sort all edges in the graph:

```
qsort(graph->edges, graph->E, sizeof(Edge), compareEdges);
```

2. Identify which vertices are the optional Steiner points:

```
for (int v = 0; v < V; v++) {
 if (!graph->is_required[v]) {
 steiner_indices[num_steiner++] = v;
 }
 }
```

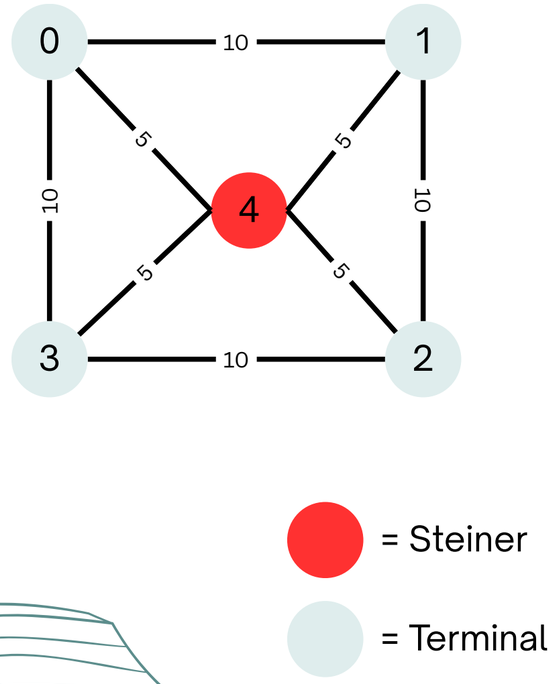
3. Generate power set:

```
// total combinations is 2^num_steiner, use bitwise operations for simpler solutions
 int total_combinations = 1 << num_steiner;

 // brute force loop generating the power set
 for (int i = 0; i < total_combinations; i++)
```

Brute Force Setup

Test Graph



4. Per element in the power set, always require terminal nodes and use bitwise operation to determine Steiner points to actvate:

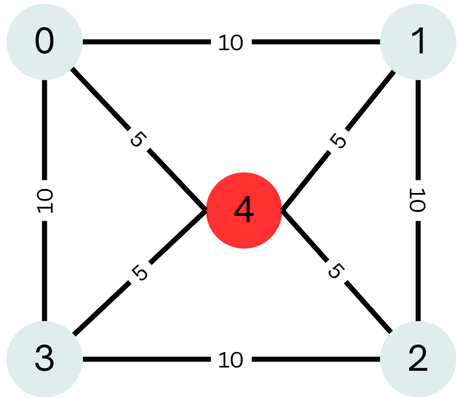
```
// reset active state: Terminals are ALWAYS on.
    for(int v = 0; v < V; v++) {
      graph->is_active[v] = graph->is_required[v];
    }

    // use bitwise AND to check the bits of our counter 'i'.
    // if the j-th bit is 1, turn on the j-th Steiner point.
    for (int j = 0; j < num_steiner; j++) {
      if (i & (1 << j)) {
        graph->is_active[steiner_indices[j]] = true;
      }
    }
```

Example for 2 Steiner points, 2^2 = 4 combinations.
Iteration 1: 00, no steiner points will be activated
Iteration 2: 01, one stiener point will be activated
Iteration 3: 10, one stiener point will be activated
Iteration 4: 11, two steiner points will be activated

Brute Force Setup

Test Graph



= Steiner

= Terminal

5. Run each element in the power set on Kruskal-based MST algorithm:

```
// run MST on this specific subset
int current_weight = activeSubsetMST(graph);
```

6. After each element is ran through the MST algorithm, update the global cheapest path:

```
// update global minimum
if (current_weight < min_total_weight) {
    min_total_weight = current_weight;
  }
```

7. Return the cheapest path:

```
return min_total_weight;
```

Output:
--- SANITY CHECK ---
Brute Force Result: 20 (Expected: 20)